54. Capstone: Broken Box Challenge

54. Capstone: Broken Box Challenge

Phase: Phase 12 — Capstones
Estimated time: 2–4 days
Slug: `54-capstone-troubleshoot`

Learning objectives

- Debug a deliberately broken VM
- Capture timeline and root cause
- Fix with minimal change
- Write postmortem

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Real systems rarely fail in one clean way. A disk fills, then a service cannot write, then a health check fails, then someone "fixes" it by changing permissions and creates a second fault. This capstone gives you a deliberately broken VM so you can practice the full loop: diagnose, repair with the smallest change, and write up what happened. The write-up matters as much as the fix; without it, the same failure comes back.

A postmortem has a small set of required sections: summary and impact, timeline, **root cause**, resolution, and prevention. If you must be strict about one, it is the root cause, because without it the document is only a diary and nothing can improve.

Core concepts

Postmortem section	Purpose
Summary and impact	who was affected, how badly, for how long
Timeline	timestamped facts: detect, diagnose, mitigate, resolve
Root cause	the underlying reason, not the trigger or the symptom
Resolution	exactly what restored service
Prevention	concrete actions with owners and dates
Lessons	what went well, what was lucky

```
symptom (what users saw)   "site returns 503"
      |   why?
proximate cause            "app cannot write its state file"
```

```
      |   why?
  deeper cause               "filesystem is 100 percent full"
      |   why?
  ROOT CAUSE                 "log rotation disabled by an edit last week"
      |
  prevention: restore logrotate, alert at 90 percent disk, add test
```

Keep asking “why” until the answer is something you can fix permanently. A person’s mistake is rarely a root cause; the missing guardrail is.

Worked examples

Example 1: a broken-box checklist, in order.

```
systemctl --failed
journalctl -p err -b --no-pager | tail -30
df -h; df -i
ip -br a; ip r
getent hosts example.com
mount | grep -E 'ro,|errors'
```

Example 2: bad fstab entry, system drops to emergency mode.

```
journalctl -xb | grep -i fstab
sudo cp /etc/fstab /root/fstab.bad
sudo nano /etc/fstab     # comment out or fix the bad line
sudo mount -a && echo MOUNT_OK
```

Example 3: permissions broken on a service directory.

```
sudo journalctl -u app.service -n 20 --no-pager
namei -l /var/lib/app/state.db
sudo chown -R app:app /var/lib/app
sudo chmod 750 /var/lib/app
```

Fix the specific path, not `chmod -R 777 /`.

Example 4: capture evidence before changing things.

```
mkdir -p ~/incident-$(date +%F) && cd ~/incident-$(date +%F)
df -h > df.before; ps aux > ps.before; journalctl -b --no-pager > journal.txt
```

Example 5: a postmortem skeleton.

```
Title: App 503 after disk fill (2026-10-21)
Impact: 38 minutes of errors for all users
Timeline:
  14:02 alert  14:07 df shows 100 percent  14:21 logs freed  14:40 verified
Root cause: logrotate disabled by manual edit; no disk alert existed
Resolution: re-enabled logrotate, freed space, restarted app
Prevention: alert at 90 percent (owner: sam, Fri), config review rule
```

Mini-lab

1. Snapshot a healthy VM. Have a friend (or you, earlier) inject at least three faults: full disk, bad fstab, wrong permissions, broken DNS, failed unit, expired cert.
2. Start a timeline file immediately; add timestamps as you go.
3. Run the baseline checks and save outputs before any change.
4. Fix one fault at a time and verify after each.
5. Do not rebuild the VM; debugging is the exercise.
6. Write the postmortem with all five sections and ask "why" at least three times to reach the root cause for each fault.
7. List prevention items that are specific, owned, and testable.

Decision guide / failure modes

Situation	Move
Many symptoms	find the earliest in the timeline
Tempting big hammer	stop; list the smallest reversible change
Fix works, cause unknown	keep investigating; it will recur
Blame-heavy write-up	rewrite around missing guardrails
Prevention is vague	add owner, date, and a check that proves it

Failure modes: recreating the VM instead of debugging; changing everything at once; no evidence preserved; postmortems that stop at the trigger ("someone ran rm"); and prevention items like "be careful". The rubric is structured troubleshooting, minimal blast radius, a clear root cause, and testable prevention.

Common mistakes

- Recreating VM instead of debugging
- Changing everything at once
- No postmortem

Exercises

Work in `exercises/54-capstone-troubleshoot/`. Read that folder's `README.md` fully before coding.

```
cd exercises/54-capstone-troubleshoot
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement `postmortem_required_section`. It prints the name of the one postmortem section the lesson calls mandatory because it explains the underlying reason an incident happened, not just the symptom or trigger. Output is lowercase,

hyphenated, one line, no extra text. After it passes, run the capstone: break three things in a snapshot, fix them one at a time with evidence, and write the postmortem with all five sections. Use at least three “why” steps for each fault and list prevention items with an owner and a date. Keep the timeline file you started at the first minute.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/54-capstone-troubleshoot/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What was root cause?
2. What evidence?
3. What prevention?

Further reading

- https://sre.google/sre-book/postmortem-culture/